ENTWICKELN EINES PROGRAMMS ZUR SIMULIERUNG VON BAUSTEINEN DES LEBENS

Maturaarbeit von Jonathan Goth

JONATHAN GOTH, 4A, GYMNASIUM BÄUMLIHOF
BETREUUNGSPERSON: DR. ANDREAS LEUTHARDT
KOREFERENT: ADRIAN KAISER
BASEL, 11.10.2026

Inhaltsverzeichnis

1 Einleitung 3
2 Prozess 4
2.1 Programmiersprache 4
2.2 Grafik-Hilfen 5
2.3 Grafikteil 5
2.4 Simulationsteil 6
2.5 Bedienungshilfen 7
2.6 Unterstützung durch KI 7
3 Produkt 8
3.1 Simulationselemente 8
Partikel 8
Verbindung 8
3.2 Globale Variablen 9
3.3 Simulationsfläche 9
3.4 Globale Uhr 9
3.5 Kräfte 11
3.6 Struktur 12
3.7 Installation 14
3.8 Kompilieren und Ausführen 15
3.9 Zukunft 15
4 Fähigkeiten des Programms 16
4.1 Phospholipid 16
Typ 1 16
Typ 2 17
4.2 DNA/RNA 18

4.3 Protein 19
4.4 Bedienung 20
5 Limits/Probleme 20
6 Fazit 21
7 Glossar 22
8 Quellen 23
8.1 Online-Archiv 23
8.2 Websites 23
8.3 Bildquellen 25
8.4 KI 25
9 Danksagung 25
10 Redlichkeitserklärung 26

1 Einleitung

Ich interessiere mich seit meiner Kindheit für Naturwissenschaften und seit einigen Jahren auch für das Programmieren. Es fasziniert mich besonders, diese beiden Interessen zu kombinieren und mit den entstehenden Programmen zu versuchen physikalische, chemische und biologische Prozesse zu simulieren. Bei der Recherche zu solchen Simulationen bin ich auf das Konzept der «Emergenz» gestossen. Es beschreibt das Auftreten von komplexen Eigenschaften eines Systems, dessen Bestandteile diese Eigenschaften nicht besitzen. Wegen seiner Unvorhersehbarkeit ist es eines der am häufigsten simulierten Phänomene mit vielen verschiedenen Beispielen. Zwei davon interessieren mich besonders und haben mich auch dazu inspiriert, dieses Programm zu schreiben.

Das Game of Life (LifeWiki contributors, 2025), entwickelt 1970 vom Mathematiker John Conway (Bhargava, Gabai, Kochen, & Kohn, kein Datum), ist das vermutlich bekannteste und einfachste Beispiel für Emergenz. Es besteht aus einem unendlich grossen zweidimensionalen Raster aus Quadraten gleicher Grösse. Jedes stellt eine Zelle dar und hat einen von zwei Zuständen, lebendig oder tot. Entstehung, Überleben und Tod einer Zelle hängt vom Zustand ihrer (acht) Nachbarn ab:

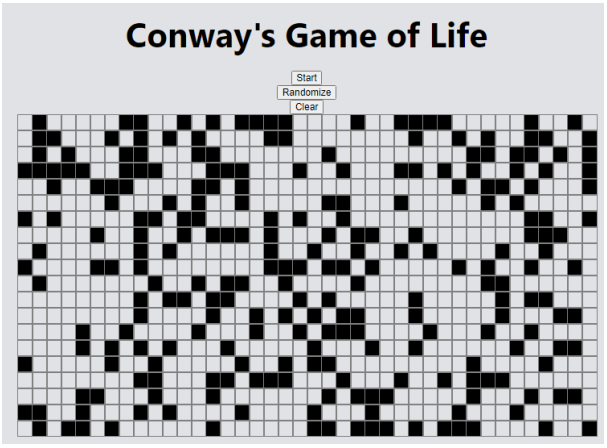


Abbildung 1: Conway's Game of Life, Schwarze Zellen sind lebendig, weisse sind tot

1. < 2 lebendige Nachbarn → tot.
2. > 4 lebendige Nachbarn → tot.
3. 2 lebendige Nachbarn → bleibt am Leben, wenn lebendig (sonst tot).
4. 3 lebendige Nachbarn → lebendig.

Die Simulation läuft in Schritten bzw. Generationen ab, wobei die lebendigen Zellen einer Generation von denen der vorherigen Generation abhängig sind. Mit diesen einfachen Regeln können riesige, organisch aussehende Gebilde entstehen, die explosionsartig wachsen und in den meisten Fällen bald darauf wieder ähnlich schnell zusammenfallen. Eine weitere Charakteristik des Game of Life ist, dass eine einzelne, falsch platzierte Zelle ein gesamtes System innerhalb weniger Generationen komplett zerstören kann.

Das zweite Beispiel ist Particle Life (Mohr, Particle Life Simulator, kein Datum), welches 2022 vom Softwareentwickler Tom Mohr (Mohr, About Me, kein Datum) auf Basis von «Clusters» (Ventrella, An asymmetrical particle system, 2025) des digitalen Künstlers Jeffrey Ventrella (Ventrella, jeffrey ventrella, kein Datum) entwickelt wurde. Es besteht ebenfalls aus einer zweidimensionalen Fläche, die jedoch eingegrenzt ist und auf der sich «Partikel» (farbige Kreise) frei bewegen. Diese Partikel sind nach ihrer Farbe in Gruppen eingeteilt, anhand derer ihre Interaktionen definiert sind. Dazu hat jede der n Gruppen n Skalare (einen für jede Gruppe → n x n Matrix), der die Stärke der Interaktion definiert. Anhand dieses Wertes erfährt ein Partikel ab einer gewissen Distanz zu anderen Partikeln eine Kraft. Diese Kräfte können anziehend oder abstossend sein und müssen nicht symmetrisch sein. Das heisst, dass Anziehung-Anziehung und Abstossung-Abstossung sowie Anziehung-Abstossung mit gleichen oder unterschiedlichen Magnituden möglich sind. Mit den richtigen Werten und genügend Zeit, können in Particle Life Gebilde entstehen, die sowohl in ihrem Aussehen als auch ihrem Verhalten Lebensformen wie Einzellern oder Würmern ähneln.

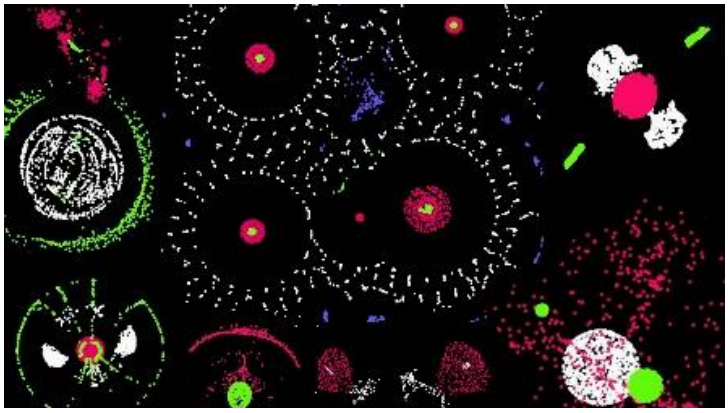


Abbildung 2: Eine Zusammenstellung von einigen Strukturen, die in Particle Life entstehen können

Diese beiden Programme haben mich dazu inspiriert, etwas ähnliches zu erschaffen.

Das Ziel meines Programms sollte es sein, organische Komponenten im Rahmen einiger Limitierungen so realitätsgetreu wie möglich zu simulieren. Gleichzeitig wollte ich damit schlussendlich auch versuchen, sehr rudimentäre Zellen mit mehreren Bestandteilen zu simulieren.

2 Prozess

2.1 Programmiersprache

Mit der Entscheidung, welche Funktionen das Programm haben sollte, wurde klar, dass Rechenleistung vermutlich die wichtigste Ressource werden würde.

Die erste und möglicherweise wichtigste Entscheidung diesbezüglich war die Wahl der zu verwendenden Programmiersprache. Mit maximaler Effizienz als Ziel kamen nur eine Handvoll Sprachen infrage, namentlich: Assembly, C, C++, Rust, Zig und Odin. Einige davon wurden jedoch schnell wieder verworfen. Diese waren: Assembly, da der Code zu komplex geworden und nicht mit allen Systemen kompatibel gewesen wäre, sowie Zig und Odin, weil beide zu neu sind und ihre Dokumentation sowie ihre

Anwendungsmöglichkeiten noch sehr limitiert sind. Von den übrig gebliebenen Sprachen war C++ die beste Wahl, da es (neben C) seit langem der Standard für grafische Programme und Games/Simulationen ist und sich wegen seinen objekt-orientierten (blau markierte Begriffe sind im Glossar, Kapitel 7 erklärt) Fähigkeiten optimal eignete. Weiterhin hatte ich mit C++ bereits über 1,5 Jahre Erfahrung und wollte nicht noch eine neue Programmiersprache lernen.

2.2 Grafik-Hilfen

Da geplant war, dass die Simulation nur einfache geometrische Formen beinhalten sollte, wäre eine existierende, gut dokumentierte Grafik-Library übertrieben gewesen und hätte möglicherweise die Geschwindigkeit beeinträchtigt. Deshalb wurde schlussendlich auch der grafische Teil des Programmes selbst geschrieben in der Hoffnung, nur die benötigten Elemente hinzuzufügen und keine Rechenleistung an nicht verwendeten Code zu verlieren. Bei einer existierenden Library könnte dies passieren, wenn sie für verschiedene Anwendungen gemacht ist und Code enthält, der für die Simulation nicht gebraucht und trotzdem ausgeführt wird. Die Wahl fiel schlussendlich auf die Vulkan Grafik API (Khronos Group, kein Datum-b). Veröffentlicht 2016 von der Khronos Group (Khronos Group, kein Datum-a) wuchs sie schnell zu einer der am häufigsten verwendeten Grafik APIs für Games bzw. Game Engines und Simulationssoftware und ist mit ihrer hohen Effizienz und Kompatibilität perfekt für dieses Programm geeignet. Das Fenster, in dem die Simulation läuft, muss zusätzlich erstellt und mit Vulkan verlinkt werden. Dazu wurde ursprünglich GLFW (Geelnard & Löwy, kein Datum) verwendet. Es wurde jedoch, bei dem Versuch einen Fehler im Programm zu finden, durch SDL3 (libsdl-org, kein Datum) ersetzt.

2.3 Grafikteil

Der erste Entwicklungsabschnitt war die grafische Komponente des Programms und hat, angefangen Ende Januar 2026, etwa drei Monate gedauert. Grundlage für dessen Code war das Vulkan Tutorial von Alexander Overvoorde (Overvoorde, 2016), welches jedoch bereits veraltet war und nicht mehr dem aktuellen Vulkan Standard entsprach. Deshalb ist davon abzuraten, es weiterhin zu verwenden. Nichtsdestotrotz konnte das Grafiksystem nach seiner Fertigstellung gegen Ende April 2026 problemlos mehrere zehntausend Kreise darstellen. Zu Beginn der Entwicklung wurde der Grafikteil in zwei Klassen aufgeteilt. Die `VulkanHandle`-Klasse abstrahiert Interaktionen mit Vulkan, wie beispielsweise die Initialisierung und soll es einfacher machen, damit zu arbeiten. Ausserdem sollte sie Fehler, die während der Vulkan Initialisierung auftreten, erkennen und ausgeben, was es einfacher macht, Fehler im Grafikteil zu finden. Die zweite Klasse, `Window`, ist für spezifischere Aktionen, wie das Zeichnen von Formen zuständig und interagiert dafür mit einer `VulkanHandle`-Instanz. Der Grossteil des Vulkan Tutorials wurde in die `VulkanHandle` Klasse übernommen und nur leicht abgeändert. Danach wurden Listen für die zu zeichnenden Elemente in die

`Window`-Klasse integriert. Jede Liste speichert Elemente einer bestimmten Form und zu Beginn wurden Listen für Kreise und Rechtecke implementiert, was, wie sich zum Schluss herausstellte, alle Formen waren, die die Simulation braucht. Die Idee dahinter ist, dass alle Formen, die gezeichnet werden sollen, temporär abgespeichert und dann als Gruppe an Vulkan übergeben werden, statt dies für jede Form einzeln zu tun. Der Grund dafür ist, dass diese Methode sehr effizient ist und um einiges schneller laufen sollte als die Alternative. Die Methoden `DrawCircle()` sowie `DrawRectangle()` der Window-Klasse fügen ein neues Element zur entsprechenden Liste hinzu und `Update()` zeichnet alle Elemente der Listen und leert sie danach.

2.4 Simulationsteil

Der eigentliche Simulationsabschnitt wurde im Mai begonnen und war Ende Juni grösstenteils fertig. Zu Beginn war noch nicht klar, wie detailliert die biologischen Elemente simuliert werden sollten. Im Austausch mit Herrn Leuthardt wurde klar, dass es sinnvoll wäre, sie sehr abstrakt und nur in ihrer Form und Funktion zu simulieren. Dies würde die Entwicklung des Programms vereinfachen und das Berechnen von chemischen Vorgängen stark reduzieren. An diesem Punkt besass das Programm bereits die Möglichkeit, einzelne Partikel und kleinere Verbindungen zu simulieren, welche man mit einem modifizierten Partikel an der Mausposition herumschieben konnte. Das erste Ziel, welches schlussendlich auch die ersten paar Wochen beanspruchte, war es, Phospholipide zu simulieren. Die grundlegende Form und Bewegung waren schnell implementiert, jedoch gab es noch Probleme mit der Interaktion, besonders sobald sich zwei Lipide berührten. Das Problem der ersten Ansätze war, dass anziehende Kräfte mit zunehmender Nähe stärker wurden und auch noch auf sich berührende Lipide wirkten. Kombiniert mit der Kollisionserkennung führte dies dazu, dass Phospholipide zu rotieren begannen und sich nicht korrekt verbanden. Die Lösung war, eine Methode zur Kräfteberechnung zu verwenden, bei der auf sich berührende Lipide keine Kräfte mehr auswirken. Nach einiger Recherche wurde dafür das Lennard-Jones Potential (Wikipedia contributors, 2026), eine Annäherung zur Berechnung von Wechselwirkungen von nicht chemisch gebundenen Atomen bzw. Teilchen, verwendet, was das Problem löste. In der letzten Woche vor der geplanten Fertigstellung des Programms wurden schliesslich noch DNA, RNA, Proteine für Transkription sowie Translation und die Möglichkeit, weitere Elemente zu erschaffen, hinzugefügt. Um die Bedienung der Simulation zu verbessern, wurde ein Speichersystem erstellt, dass den aktuellen Zustand der Simulation als JSON-Datei speichern und laden kann. Ausserdem wurde noch ein Partikel hinzugefügt, mit dem man Elemente löschen kann, statt sie zu bewegen. Ab diesem Zeitpunkt war die Simulation komplett funktionsfähig.

2.5 Bedienungshilfen

Anfang Oktober, kurz vor der Abgabe, wurden noch zwei grosse Veränderungen am Programm vorgenommen, um die Bedienung klarer und einfacher zu gestalten. Zum Einen wurde das Verändern und Hinzufügen von Verbindungen vereinfacht. Bis dahin musste das Programm nach jeder Veränderung der Verbindungen neu kompiliert werden, was spezielle Programme und Wissen benötigt. Hinzugefügt wurde die Möglichkeit, Verbindungen in einer externen Datei zu definieren, welche vom Programm gelesen wird, was dies umgeht. Um Veränderungen anzuwenden, muss das Programm nur noch neu gestartet werden. Die zweite Bedienungshilfe ist ein Interaktionsmenü im Terminal. Dabei können sowohl beim Ausführen des Programms, als auch während es läuft, bestimmte Parameter ausgegeben oder verändert werden. Dies ermöglicht es, die Simulation zu verändern, während das Programm läuft.

2.6 Unterstützung durch KI

Während des gesamten Entwicklungsprozesses des Programms wurde ChatGPT als Hilfe hinzugezogen. Dabei war dessen Nutzung jedoch strikt darauf beschränkt, neue Konzepte zu erklären, anhand eines Beispiels die Nutzung einer Fähigkeit von C++ oder einer Library zu zeigen oder besonders hartnäckige Fehler im Code zu finden. Ohne diese Hilfe hätte die Entwicklung des Programms viel mehr Zeit beansprucht und die Simulation wäre nicht in ihrem jetzigen Ausmass möglich gewesen. Wichtig ist jedoch, dass der gesamte Code des Programms entweder selbst geschrieben oder aus einem Tutorial entnommen wurde.
Beim Schreiben der Dokumentation wurde ChatGPT nur zweimal zur einheitlichen Erklärung einiger Begriffe verwendet. Beide Abschnitte wurden manuell überprüft und die Nutzung von ChatGPT ist jeweils angegeben (Kapitel 3.5 und 7).

3 Produkt

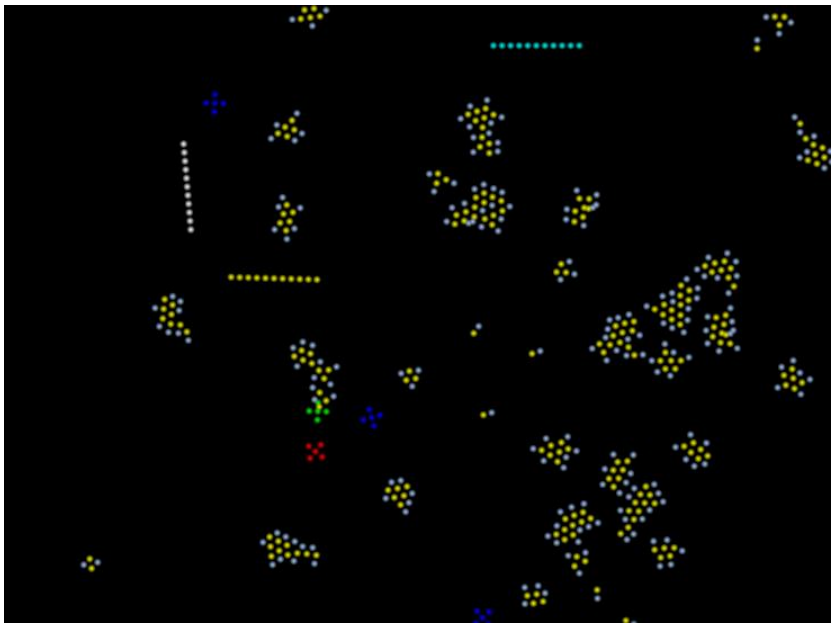


Abbildung 3: Ein zufälliges Beispiel der Simulation mit verschiedenen Verbindungen

3.1 Simulationselemente

Die Elemente der Simulation sind in zwei Gruppen unterteilt:

Partikel

Partikel sind die kleinsten Bausteine der Simulation bestehend aus einem Kreis mit einem Radius (in Nanometern) und einer Farbe. Sie können einzelne Atome oder kleinere Moleküle darstellen und sind entweder hydrophil oder hydrophob. Jedes Partikel in der Simulation ist Teil einer Verbindung.

Verbindung

Verbindungen sind Gruppen aus Partikeln, die eine Funktionseinheit wie z.B. ein Molekül, Protein, Enzym oder DNA/RNA. bilden. Dabei bleibt die Struktur statisch, bzw. die Partikel haben zueinander eine feste Position. Verbindungen interagieren mit anderen Verbindungen über hydrophile und hydrophobe Wechselwirkungen und können bei physischem Kontakt zuvor definierte Funktionen ausführen, was neue Interaktionen ermöglicht.

3.2 Globale Variablen

Um das Organisieren des Codes in mehrere Dateien, das Speichern einer Simulation und das Verändern von Parametern der Simulation zu vereinfachen, sind viele der Variablen, die während der gesamten Laufzeit benötigt werden, als globale Variablen implementiert. Das heisst, dass sie von allen Stellen im Code aus erreichbar und veränderbar sind.

3.3 Simulationsfläche

Die Simulation ist zweidimensional und auf eine bestimmte Grösse limitiert. Diese Fläche ist weiss umrahmt und hat eine Standardgrösse von 100 Nanometer x 100 Nanometer. Ihre Position und Grösse wird durch die globale Variable `simArea` vorgegeben. Diese ist ein Vierdimensionaler Vektor, dessen Komponenten wie folgt interpretiert werden:

`X` → Die x-Position des linken oberen Ecks der Fläche.
`Y` → Die y-Position des linken oberen Ecks der Fläche.
`Z` → Die Breite der Fläche in positive x-Richtung.
`W` → Die Höhe der Fläche in positive y-Richtung.

Daraus ergibt sich, dass die ersten beiden Komponenten die Position des linken oberen Ecks der Fläche bestimmen und die anderen ihre Grösse.
Bei der Simulation wird anhand des Schwerpunkts bzw. dem Massezentrum aller Partikel einer Verbindung geprüft, ob sie sich noch innerhalb der Simulationsfläche befindet. Ist dies nicht der Fall, wird versucht, die Verbindung durch Umdrehen der x– oder y-Komponente ihrer Geschwindigkeit wieder in die Simulationsfläche zu drücken. Dies erzeugt einen Abpralleffekt, sobald eine Verbindung eine Wand der Fläche berührt. Falls sich eine Verbindung ohne Geschwindigkeit ausserhalb der Fläche befindet, bleibt sie dort stecken, bis sie von aussen in Bewegung versetzt oder gelöscht wird.

3.4 Globale Uhr

Bei Computerprogrammen gibt es zwei verschiedene Zeitfenster, die teilweise unabhängig voneinander ablaufen. Das erste ist aus Sicht des Betrachters bzw. die benötigte Zeit zur Berechnung eines Frames und das zweite aus Sicht des Programms bzw. die Zeit, die während dieses Frames simuliert wird. Für eine Simulation mit 10 FPS (Frames pro Sekunde) müsste im Programm pro Frame 0.1 Sekunden verstreichen, damit beide Zeitfenster gleich sind.

Um die Abläufe so realitätsgetreu wie möglich darzustellen, muss die Zeit innerhalb der Simulation linear ablaufen. Das heisst, dass in jedem Frame die gleiche Zeit verstreichen muss. Da mit zunehmender Komplexität die Zeit zur Berechnung eines Frames steigt, scheint sich die Geschwindigkeit des Programms zu verändern. Dies wird normalerweise durch das Anpassen der Simulationszeit anhand der Berechnungszeit ausgeglichen, führt aber auch zu ungleichmässigen Zeitsprüngen in der Simulation und ist hier ungeeignet. Deshalb deckt sich die Simulationszeit nicht mit der Zeit in Wirklichkeit, weshalb es auch nicht möglich ist, die verstrichene Zeit in der Simulation von aussen zu messen. Um dieses Problem zu lösen, hat die Simulation eine eigene Uhr, die die Simulationszeit misst.

Standardmässig ist die Simulationszeit pro Frame auf 0.001 Sekunden (1 Millisekunde) gesetzt und von der globalen Variable `simFrameTime` vorgegeben. Somit ist in der Simulation nach 1000 Frames 1 Sekunde verstrichen. Um die Zeit nach einem beliebigen Frame zu berechnen, muss die Anzahl verstrichener Frames seit Simulationsanfang gezählt werden, wofür die globale Variable `frameCounter` zuständig ist. Mit diesen beiden Variablen lässt sich die verstrichene Simulationszeit berechnen:

$$frameCounter \times simFrameTime = Verstrichene\ Simulationszeit$$

Jedoch gibt es auch die Möglichkeit, die Framezeit zu verändern, was es schwieriger macht, die Simulationszeit zu messen. Deshalb wird jedes Mal, wenn die Framezeit verändert wird, die verstrichene Zeit seit der letzten Veränderung (oder dem Start der Simulation, wenn die Framezeit noch nicht verändert wurde) gemessen und zu potenziellen vorherigen Messungen addiert. Dies ermöglicht es, die Zeit seit Simulationsbeginn ausschliesslich bis zum aktuellen Frame zu berechnen. Um mit dieser Methode die Zeit zwischen zwei beliebigen Frames zu messen, muss die Zeit beider Frames bekannt sein, was bedeutet, dass man beim ersten Frame die verstrichene Zeit messen und bis zum Erreichen des zweiten Frames speichern muss. Die Globale Uhr ermöglicht dies durch mehrere Methoden:

`recordNow()` → Muss beim ersten Frame aufgerufen werden und speichert die aktuelle Zeit beim Start der Messung.

`read(frame)` → Kann beliebig oft aufgerufen werden und gibt die Zeit seit dem angegebenen Frame aus. Der Frame muss jedoch zuvor mit `recordNow()` gespeichert worden sein.

`deleteEntry(frame)` → Löscht den gespeicherten Frame um Speicherplatz freizugeben, wenn er nicht mehr benötigt wird.

3.5 Kräfte

Die einzigen Kräfte, die zwischen Verbindungen simuliert werden, sind hydrophile und hydrophobe Wechselwirkungen. Die Stärke der Interaktion wird dabei mit dem Lennard-Jones Potential (Wikipedia contributors, 2026), einer Annäherung zur Berechnung von Wechselwirkungen von nicht chemisch gebundenen Atomen bzw. Teilchen, berechnet.

$$V = 4\varepsilon\left\{\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right\} = \varepsilon\left\{\left(\frac{r_m}{r}\right)^{12} - 2\left(\frac{r_m}{r}\right)^{6}\right\}$$

Abbildung 4: Das Lennard-Jones Potential als Funktion

Erklärung der Parameter, generiert von ChatGPT (OpenAI, kein Datum):

V = potentielle Energie der Wechselwirkung zwischen den beiden Teilchen

→ Gibt an, wie stark sich die Teilchen anziehen oder abstossen.

r = aktueller Abstand zwischen den beiden Teilchen

→ Je nach r verändert sich die Wechselwirkungsenergie.

ε = Tiefe des Potentialtopfes bzw. Stärke der Wechselwirkung

σ = charakteristischer Abstand des Lennard-Jones-Potentials

r_m = Gleichgewichtsabstand bzw. Abstand beim Minimum des Potentials

→ Hier ist die potentielle Energie am niedrigsten und die resultierende Kraft ist Null.

Das Lennard-Jones Potential ergibt das Potential zwischen zwei Teilchen und kann durch Ableiten zu einer Kraft umgerechnet werden. Die Kurve der Ursprungsfunktion ist abhängig von der Distanz zweier Teilchen und führt zu einem negativen Potential (üblicherweise anziehend) bei einer grösseren Distanz und einem stark positiven Potential (üblicherweise abstossend) bei einer zu kleinen Distanz. Dazwischen gibt es einen Punkt, an dem sich die Potentiale ausgleichen und keine Kraft auf die Teilchen wirkt, was ihre optimale Distanz beschreibt.

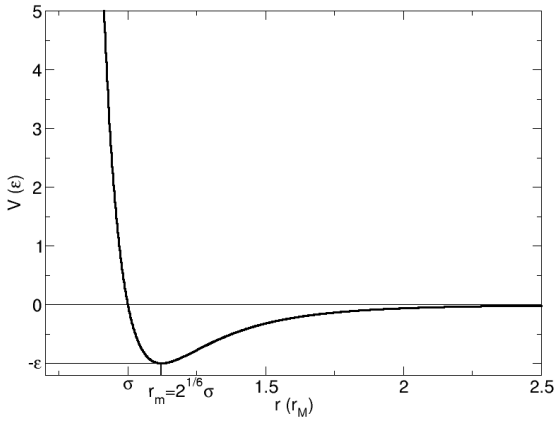


Abbildung 5: Das Lennard-Jones Potential als Kurve dargestellt

3.6 Struktur

Abgesehen von Vulkan ist der gesamte Code der Simulation und alle benötigten Libraries im Hauptordner namens `lifelike-engine` strukturiert.

```
lifelike-engine/
|-- assets/
|   |-- compound_templates.cpp
|-- doc/
|   |-- Doxyfile
|-- include/
|   |-- assets/
|   |   |-- compound_templates.hpp
|   |-- core/
|   |   |-- result.hpp
|   |   |-- types.hpp
|   |   |-- vector.hpp
|   |   |-- vector.inl
|   |-- graphics/
|   |   |-- vulkan_handle.hpp
|   |   |-- vulkan_handle.inl
|   |   |-- window.hpp
|   |-- sim/
|   |   |-- colors.hpp
|   |   |-- compound.hpp
|   |   |-- console_interface.hpp
|   |   |-- functions.hpp
|   |   |-- parameters.hpp
|   |   |-- particle.hpp
|   |   |-- save_load.hpp
|   |   |-- time_manager.hpp
|-- shaders/
|-- src/
|   |-- main.cpp
|   |-- vulkan_handle.cpp
|   |-- window.cpp
|-- third_party/
|-- CmakeLists.txt
|-- LICENSE
|-- README.md
```

`assets/compound_templates.cpp:`

Enthält alle Verbindungen, die in der Simulation erstellt werden können und alle Funktionen, die bei Kontakt zweier Verbindungen ausgeführt werden können. Durch Bearbeiten dieser Datei und Kompilation können bestehende Verbindungen verändert und neue erstellt werden.

`doc/:`

Gedacht, um hier mit Doxygen eine Dokumentation des gesamten Codes zu erstellen.

`include/assets/:`

Enthält Header für alle C++ Dateien in `assets/`, in denen alle wichtigen Elemente deklariert sind.

`include/core:`

Enthält eine Header-only Library mit Elementen, die überall im Programm verwendet werden. Die Wichtigsten sind:

Integrale Datentypen [`u8_t`, `i8_t`, `u16_t`, `i16_t`, `u32_t`, `i32_t`, `u64_t`, `i64_t`, `size_t`]

Gleitkomma Datentypen [`f32_t`, `f64_t`]

Boolean Datentyp [`b8_t`]

C-Style String [`cstr_t`]

Resultat-Enum für Debugging und Tracking von Fehlern im Grafikteil [`Result`]

Minimalistische n-dimensionale Vektor-Klasse [`Vec<2, T>`, `Vec2<T>`, `Vec3<T>`, ...]

`include/graphics:`

Enthält Header des Grafikteils, in denen die Klassen `VulkanHandle` und `Window` und deren Mitglieder deklariert sind.

`include/sim:`

Enthält Header des Simulationsteils, in denen alle restlichen Elemente der Simulation definiert sind. Deklaration und Definition sind hier in einer Datei, um es für den Compiler einfacher zu machen, das Programm zu optimieren.

`shaders:`

Enthält die von Vulkan benötigten Shader, welche in GLSL geschrieben sind und zu SPIR-V kompiliert wurden.

`src/main.cpp`:

Enthält die Funktion `int main()`, in welcher die Ausführung des Programms startet, und beinhaltet ausserdem die Hauptschleife der Simulation, welche in jedem Frame wiederholt wird.

`src/vulkan_handle.cpp & src/window.cpp`:

Enthalten die Definitionen der Header-Dateien aus `include/graphics`.

`third_party`:

Enthält alle verwendeten Libraries ausser Vulkan. Diese sind SDL3 (libsdl-org, 2026), dujingnins inicpp-Library (dujingning, 2026) und Niels Lohmanns json-Library (Lohmann, 2026). Weiterhin noch vit-vits CTPL-Library (vit-vit, 2015) für parallele Code Ausführung zu Optimierungszwecken, sie wurde jedoch (evtl. temporär) aus dem Code entfernt, um Probleme zu vermeiden.

`CMakeLists.txt`:

Enthält die CMake Konfiguration des Programms.

LICENCE:

Die MIT Lizenz (MIT, kein Datum), unter der das Programm veröffentlicht wurde und benutzt werden darf.

`README.md`:

Enthält alle nötigen Informationen, um das Programm zu installieren und zu bedienen.

3.7 Installation

Aktualisierte Informationen und Anleitungen zur Installation sind in `README.md` zu finden.

Das Programm wurde bis zur Veröffentlichung nur auf Windows 11 Home und Arch Linux getestet. Kompatibilität mit MacOS konnte hingegen nicht getestet werden. Es hat einige Abhängigkeiten, die installiert sein müssen:

- Die Vulkan SDK (LunarG; Khronos Group, kein Datum)
- Einen C++ Compiler (Linux: GCC (GCC, 2026), Windows: MSVC (Microsoft, 2026) oder MinGW (mingw-w64, 2026))
- CMake (kitware, kein Datum)

Die letzten zwei werden nur für die Kompilierung gebraucht. Dass GitHub Repository (Goth, LifeLike-Engine, 2026) beinhaltet den restlichen Code und kann entweder manuell oder per Terminal-Befehl heruntergeladen werden. Leider müssen die Libraries in `third_party` normalerweise zusätzlich

heruntergeladen werden, da sie nur Links zu den respektiven Repositories sind. Danach muss das Programm kompiliert werden (siehe Kapitel 3.8). Alternativ sind bereits kompilierte Versionen des Programms unter «Releases» (Goth, LifeLike-Engine Releases, 2026) im GitHub Repository verfügbar.

3.8 Kompilieren und Ausführen

Aktualisierte Informationen und Anleitungen zum Kompilieren und Ausführen sind in `README.md` zu finden.

Das Programm nutzt CMake, um den Code zu kompilieren bzw. ausführbar zu machen, dies funktioniert aktuell jedoch nur auf Linux. Um Ordnung zu behalten, sollte für die Kompilation ein neuer Ordner erstellt werden. Ausgehend vom Hauptordner (der CMakeLists.txt enthält) kann man es mit den folgenden Terminal-Befehlen kompilieren.

```
1. mkdir build && cd build
2. cmake ..
3. cmake --build . --config Release
```

Wenn keine Fehler auftreten, sollte sich das kompilierte Programm mit dem Namen `LifeLike-Engine` im build-Ordner befinden. Manche Compiler, wie MSVC, der standard C++ Compiler auf Windows, platzieren die Datei möglicherweise in einen Unterordner.

3.9 Zukunft

Da das Programm nach der Veröffentlichung vermutlich einige Updates erhält, können sich einige Elemente noch ändern. Anleitungen zur Installation und Nutzung sowie weitere wichtige Informationen sind in README.md zu finden und werden zeitnah angepasst.

4 Fähigkeiten des Programms

Die Simulation kann diverse Zellbestandteile und chemische Verbindungen in ihrer Form und ungefähren Verhaltensweise simulieren. Hydrophobe und hydrophile Wechselwirkungen nach dem Lennard-Jones Potential reichen in manchen Fällen bereits aus, um komplexere Gebilde aus mehreren Verbindungen entstehen zu lassen. Dazu können mit beliebig vielen Aktionen beim Kontakt zweier Verbindungen komplexere Interaktionen ermöglicht werden. Hier sind einige Beispiele, die implementiert und getestet wurden.

4.1 Phospholipid

Die ersten und einfachsten Beispiele für eine Verbindung sind Phospholipide, die sich unter den richtigen Bedingungen und nach einiger Zeit zu kurzen Abschnitten einer Doppelmembran zusammenschliessen können.

Typ 1

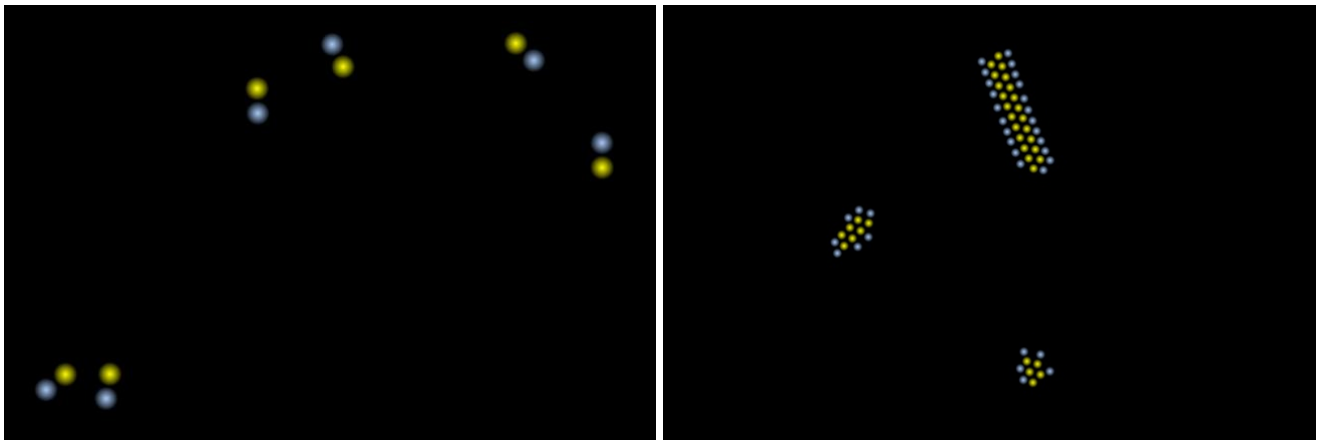


Abbildung 6: (Links) Einzelne Phospholipide vom Typ 1. (Rechts) Eine Kette aus Phospholipiden

Jedes Phospholipid besteht aus einem hydrophilen (blau) und einem hydrophoben (gelb) Partikel. Durch starke hydrophobe Anziehung entstehen Gruppen aus Phospholipiden, deren hydrophobes Ende nach innen und das hydrophile Ende nach Aussen zeigt. Mit den richtigen Werten für Kräfte zwischen hydrophob-hydrophob, hydrophob-hydrophil, hydrophil-hydrophil und für die Distanz, bei der diese auftreten, bilden sich längere Abschnitte bestehend aus zwei Reihen aus Phospholipiden. Analog zu einer typischen Zellmembran liegen dabei die hydrophilen Enden beider Reihen aussen. Diese Abschnitte bleiben jedoch normalerweise kurz und können sich nicht zu einer längeren Kette verbinden. Der Grund dafür ist

wahrscheinlich die limitierte Bewegungsfreiheit im zweidimensionalen Raum. Sie führt dazu, dass sich die Phospholipide an den Enden der Ketten nicht in der gleichen Orientierung befinden, wie der Rest und relativ frei rotieren können und oft so ausgerichtet sind, dass sie den Zugang zum hydrophoben inneren Teil der Kette blockieren. Somit prallen zwei «offene» Ketten fast immer voneinander ab, auch wenn sie sich an den Enden berühren. Ein Lösungsansatz wäre es, die Distanz, auf der die anziehenden Kräfte wirken zu verringern, jedoch führt das auch dazu, dass vereinzelte Phospholipide an beliebigen Stellen der Kette haften bleiben und deren Struktur destabilisieren, bis sie zusammenfällt und sich die Phospholipide wieder in kleinere Gruppen oder grosse unordentliche Komplexe zusammenziehen. Durch das Verstärken der hydrophil-hydrophil Anziehung würden die Lipide an den Enden der Ketten teilweise in die richtige Orientierung versetzt werden, jedoch führt dies auch dazu, dass sich wieder grössere Klumpen bilden, in denen die Struktur der Ketten zerstört wird.

Typ 2

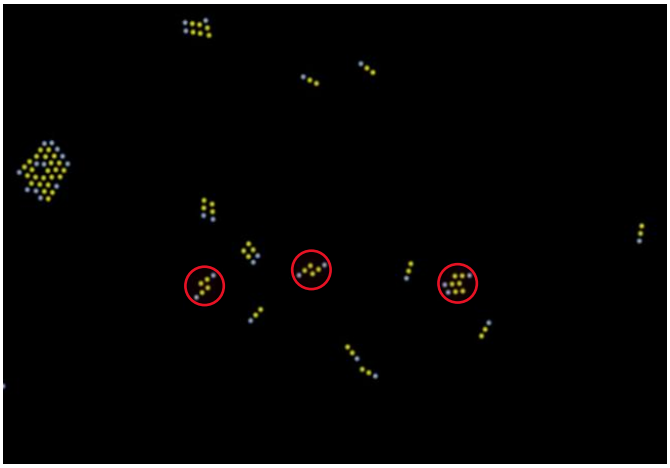


Abbildung 7: Phospholipide vom Typ 2. Rot umkreiste Gruppen sind nicht mehr in der Lage, eine Doppelmembran zu bilden

Ein weiterer Lösungsansatz ist, die Phospholipide zu verlängern, um eine zufällige/ungewollte Orientierung einzelner Lipide einer Kette zu verhindern. Deshalb gibt es im Programm noch ein zweites Phospholipid, das zwei hydrophobe und ein hydrophiles Partikel besitzt. Die beiden hydrophoben Partikel bewirken zusätzlich eine stärkere Kraft als nur ein Einzelnes. Bei den Tests hat sich jedoch ergeben, dass hier die Zweidimensionalität wieder einige Probleme erzeugt. Das grösste Problem ist die geometrisch optimale Anordnung zweier dieser Phospholipide. Sie führt dazu, dass es unwahrscheinlicher ist, dass eine Doppelmembran entsteht und ausserdem keine Membran mit glatter Oberfläche mehr möglich ist. Der Grund dafür ist einerseits, dass sich zwei Lipide wegen der stärkeren Anziehung praktisch nicht mehr voneinander trennen lassen, sobald sie sich angezogen haben und andererseits, dass sich zwei Lipide wegen

ihrer Geometrie nie perfekt nebeneinander positionieren, sondern immer etwas verschieben. Deshalb sind Lipide, die in entgegengesetzter Orientierung aneinander liegen, praktisch unbrauchbar und die Lipide in Ketten sind so stark aneinander gebunden, dass die Kette bei Krafteinwirkungen von aussen brechen kann statt sich zu verbiegen. Aus diesen Gründen scheinen die Phospholipide Typ 2 für eine funktionierende Zellmembran ungeeignet zu sein.

4.2 DNA/RNA

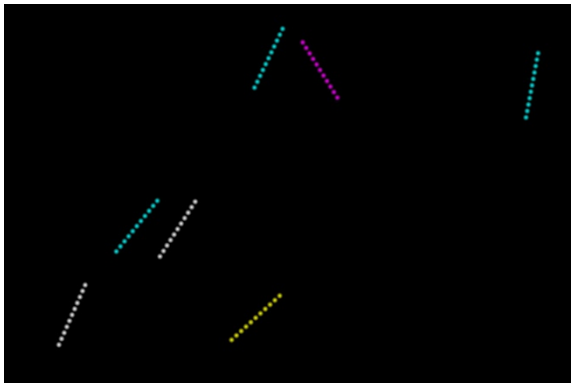


Abbildung 8: Beispiele von verschiedenen DNA- und RNA-Strängen

Eine weitere einfache Verbindung sind DNA und RNA-Stränge, die beide exakt gleich aufgebaut sein können. Standardmässig besitzt die Simulation bereits ein System, das Interaktionen zwischen DNA/RNA und Proteinen simuliert, um Transkription und Translation zu ermöglichen. Jedoch ist dieses System nur eine von vielen möglichen Herangehensweisen und folgende Beschreibungen zur Transkription und Translation beziehen sich nur auf diese eine Methode. Die Reihenfolge der Partikel einer DNA/RNA Verbindung gibt die Transkriptions- und Translationsrichtung an. Üblicherweise ist das erste Partikel am einen und das letzte am anderen Ende des Stranges, was bedeutet, dass Proteine in diese Richtung an ihm entlang wandern. Ist das Ende erreicht, wird eine neue Verbindung erstellt. Jeder DNA/RNA Strang besitzt eine Liste an Verbindungen, die sie beschreibt. Bei Beendung der Transkription/Translation wird ein zufälliges Element dieser Liste ausgewählt und erstellt. Somit kann aus der Zusammensetzung der Liste eine Wahrscheinlichkeit für ein bestimmtes Produkt angegeben werden, womit beispielsweise Translationsfehler beziehungsweise die Bildung von defekten Proteinen mit einer bestimmten Wahrscheinlichkeit simuliert werden können. Durch das Anpassen der räumlichen Struktur des Stranges können zusätzlich Interaktionen ermöglicht oder verhindert werden. Beispielsweise würde eine genügend grosse Lücke es unwahrscheinlicher machen, dass kleine Proteine das Ende des Stranges erreichen, da sie kurzzeitig von ihm getrennt werden.

4.3 Protein

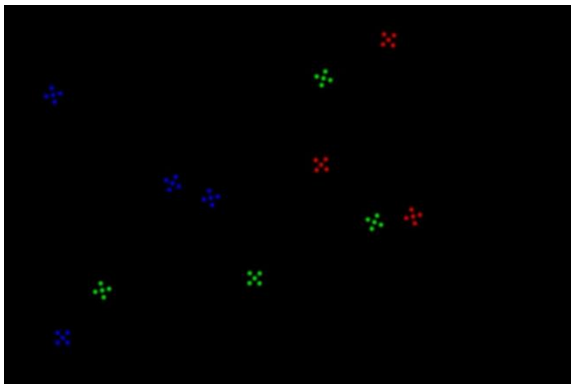


Abbildung 9: Beispiele von Proteinen mit unterschiedlichen Funktionen

Der letzte getestete Verbindungstyp sind Proteine, welche die meisten komplexen Interaktionen ermöglichen. Die Simulation enthält standardmässig Proteine, welche die Transkription und Translation ermöglichen. Auch hier sind viele verschiedene Implementationen möglich und Folgendes ist nur eine davon: Berührt ein Protein das erste Partikel einer Verbindung, mit der es interagiert, gleicht es seine Geschwindigkeit an, um mit der Verbindung in Kontakt zu bleiben. Solange dies der Fall ist, bewegt es sich langsam zum nächsten Partikel und weiter bis zum Ende der Verbindung. Berührt es das letzte Partikel, gilt dies als erfolgreiche Transkription/Translation und eine neue Verbindung wird erstellt.

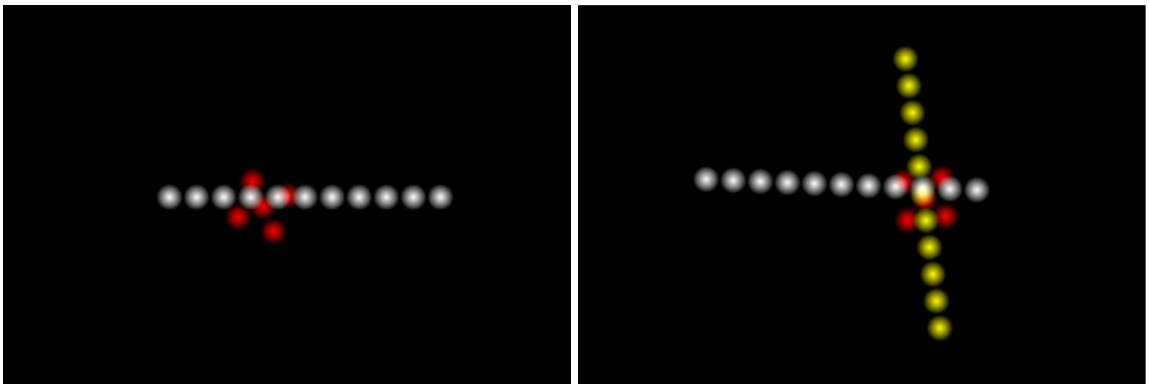


Abbildung 10: (Links) Ein Protein (Rot), das an einem DNA-Strang Transkription ausführt. (Rechts) Beim Beenden der Transkription wird ein RNA-Strang (Gelb) erstellt

4.4 Bedienung

Verbindungen können in der Datei `compounds.json` im JSON Format oder in der Liste `compounds_templates` in `assets/compound_templates.cpp` verändert oder erstellt werden. Wichtig ist, dass sich `compounds.json` im Ordner befindet, in dem das Programm danach sucht. Standardmässig ist dies `templates`, welcher sich im gleichen Ordner wie das Programm befindet. Spezielle Interaktionen können der zweiten Liste weiter unten in `assets/compound_templates.cpp` angefügt werden. Für beide gibt es standardmässig bereits Beispiele und am Ende beider Listen einen Kommentar, der die Struktur eines anzufügenden Elements erklärt. Um Änderungen in `assets/compound_templates.cpp` anzuwenden, muss das Programm neu kompiliert werden. Solange nichts am restlichen Code verändert wurde, sollte dies jedoch nicht lange dauern. Nun kann eine neue oder zuvor gespeicherte Simulation gestartet werden. Beinhaltet eine gespeicherte Simulation Verbindungen, müssen deren Interaktionsmöglichkeiten noch unter dem gleichen Namen in der Liste enthalten sein, andernfalls kann die Simulation nicht geladen werden. Die Liste mit Verbindungen kann problemlos verändert werden.

5 Limits/Probleme

Die Simulation hat viele Aspekte, die nicht der realen Welt entsprechen.
Wie bereits angesprochen, ist sie durch ihre Zweidimensionalität stark eingeschränkt und die meisten Prozesse aus der echten Welt funktionieren in 2D nicht wirklich. Deshalb müssen sie weggelassen oder umgangen werden.
Ein anderes wichtiges Problem ist, dass der erste Grundsatz der Thermodynamik gebrochen wird, da, obwohl die Simulation ein geschlossenes System darstellt, die Gesamtenergie nicht konsistent bleibt. Dies geschieht hauptsächlich durch Dämpfungsfaktoren, die zu hohe Geschwindigkeiten und Rotationen verhindern sollen.
Ein weiteres Problem ist, dass Verbindungen keine Elastizität besitzen und ihre Struktur nicht ohne Weiteres verändern können. Somit ist beispielsweise die Bildung von Chromosomen aus einem einzelnen DNA-Strang nicht möglich.
Ein weiterer limitierender Faktor ist die Geschwindigkeit der Simulationszeit, die, wie bereits angesprochen, variieren kann. Ein Lösungsansatz dafür wäre, die Frames einzeln aufzunehmen und mit der in der Simulation verstrichenen Zeit zu einem Video zusammen zu fügen. Wenn die simulierte Zeit während der Aufnahme pro Frame 0.001 Sekunden beträgt, müsste dieses Video somit eine Bildfrequenz von 1000 Bildern pro Sekunde besitzen.

Auch ist die Rechenleistung des Computers, auf dem die Simulation läuft, oftmals ein limitierender Faktor. Abhängig von der Rechenleistung verlangsamt sich die Simulation drastisch, sobald mehrere Hundert Partikel simuliert werden.
Schliesslich, auch wenn das Programm in grossen Teilen ohne Programmiererfahrung bedient werden kann, benötigt es trotzdem etwas Zeit, um die verschiedenen Elemente zu verstehen. Einige davon, wie das Bearbeiten spezieller Interaktionen sind zudem nur mit C++ Kenntnissen möglich.

6 Fazit

Das Programm entspricht im Grossen und Ganzen den Erwartungen, die ich zu Beginn der Arbeit hatte. Es kann auch auf weniger performanten Computern konsistent mehrere tausend Verbindungen und biologisch-physikalische Prozesse glaubwürdig simulieren. Während der etwa sechsmonatigen Entwicklungsphase habe ich laut meinem Arbeitsjournal etwa 200 Stunden daran gearbeitet, wobei diese Zahl wegen Ungenauigkeiten nach eigenen Schätzungen um bis zu 25% von der wirklichen aufgewendeten Zeit abweichen könnte. Diese Zeit hat nicht nur dieses Programm hervorgebracht, sondern eine grosse Menge an neuem Wissen und Erfahrung. Jedoch gibt es auch einige Ziele, die nicht erreicht wurde. Das Wichtigste ist, dass das Programm noch nie eine funktionierende Zelle simuliert hat. Das grösste bis jetzt getestete Konstrukt war eine geschlossene Zellmembran aus einigen hundert Phospholipiden. Damit verbunden ist die enorme, theoretisch unendliche Menge an möglichen Verbindungen und Interaktionen, von denen nur sehr wenige ein interessantes Ergebnis liefern. Es beansprucht viel Zeit und Versuche, auf eine funktionierende Konfiguration zu stossen. Verglichen mit der Biologie auf der Erde lassen sich daraus mehrere Schlüsse ziehen. Der Offensichtlichste ist, dass die Simulation viel zu ungenau und simpel ist, um sie mit der realen Welt zu vergleichen. Interessanter ist jedoch, wenn angenommen wird, dass zumindest einige der Prozesse in der Simulation denen der Realität entsprechen. In diesem Fall könnte man mit ihr die Entstehung von biologischem Leben erforschen und mit genügend Geduld vielleicht sogar einzelliges oder mehrzelliges Leben, aus dem «Nichts» entstehen lassen. Dies sind jedoch nur Spekulationen und ich gehe davon aus, dass das Programm viel zu langsam ist, um einen solchen Prozess zu simulieren. Jedoch bräuchte es nur einen erfolgreichen Versuch, eine Zelle entstehen zu lassen, um diese These zu widerlegen. Für mich ist das Programm ein guter erster Ansatz, mikrobiologische Systeme zu simulieren. Was mich beim Testen des Programms immer wieder erstaunt hat, ist, dass das Lennard-Jones Potenzial schon ausreicht, um diverse zwischenmolekulare Interaktionen zu simulieren. Ich plane, nach der Veröffentlichung des Programms noch einige Zeit daran weiterzuarbeiten und seine Dokumentation zu vervollständigen und hoffe, es dient als Inspiration oder Grundlage für ähnliche naturwissenschaftliche Projekte.

7 Glossar

Alle Erklärungen stammen von ChatGPT (OpenAI, kein Datum).

- **API:** Eine festgelegte Schnittstelle, über die verschiedene Softwarekomponenten miteinander kommunizieren können.
- **CMake:** Ein Programm zur automatisierten Erstellung und Verwaltung des Build-Prozesses eines Softwareprojekts.
- **Compiler:** Ein Programm, das Programmcode in eine für den Computer verständliche Form übersetzt.
- **Debugging:** Der Prozess, bei dem Fehler in einem Programm gefunden und behoben werden.
- **Definition:** Die genaue Festlegung oder Umsetzung dessen, was zuvor deklariert wurde.
- **Deklaration:** Die Bekanntgabe, dass beispielsweise eine Variable, Funktion oder Klasse existiert.
- **Enum:** Ein Datentyp, der eine feste Menge benannter Werte definiert.
- **Frame:** Ein einzelnes Bild innerhalb einer Abfolge von Bildern, die beispielsweise eine Animation ergeben.
- **GitHub Repository:** Ein Speicherort für Dateien, Quellcode und Versionsverläufe eines Projekts auf der Plattform GitHub.
- **GLFW:** Eine Bibliothek zum Erstellen von Fenstern und zur Verarbeitung von Eingaben.
- **GLSL:** Eine Programmiersprache, die hauptsächlich zur Programmierung von Shadern für die Grafikverarbeitung verwendet wird.
- **Header (C/C++):** Eine Datei, die wichtige Deklarationen und Informationen enthält, die von anderen Programmteilen verwendet werden können.
- **Header-only Library:** Eine Bibliothek, deren vollständiger Code in Header-Dateien enthalten ist und daher keine separate Implementierungsdatei benötigt.
- **Instanz:** Ein konkretes Objekt, das anhand einer Klasse erstellt wurde.
- **Klasse:** Eine Vorlage, die festlegt, welche Eigenschaften und Funktionen ein bestimmter Objekttyp besitzt.
- **Khronos Group:** Eine Organisation, die offene Standards und Schnittstellen für Grafik, Berechnung und Medien entwickelt, darunter beispielsweise OpenGL, Vulkan und OpenCL.
- **Kompilation:** Der Vorgang, bei dem Quellcode von einem Compiler in ausführbaren oder weiterverarbeitbaren Maschinencode übersetzt wird.

- **Library:** Eine Sammlung vorgefertigter Programmteile, die in anderen Programmen verwendet werden können.
- **Methode:** Eine Funktion, die zu einer Klasse gehört und das Verhalten eines Objekts beschreibt.
- **MSVC:** Eine Entwicklungsumgebung bzw. ein Compiler von Microsoft, der hauptsächlich zum Übersetzen von C- und C++-Programmen in ausführbaren Code verwendet wird.
- **Objekt-Orientiertes Programmieren (OOP):** Ein Programmierkonzept, bei dem Daten und zugehörige Funktionen in sogenannten Objekten zusammengefasst werden.
- **SDL3:** Eine Bibliothek für Fenster, Eingaben, Audio und Grafik in Programmen.
- **SPIR-V:** Ein standardisiertes Zwischenformat, das unter anderem zur Darstellung und Ausführung von Shader-Code auf Grafikkarten verwendet wird.

8 Quellen

8.1 Online-Archiv

Es existieren zwei online Archive beziehungsweise GitHub Repositories zu dieser Arbeit. Eines für die Dokumentation (Goth, Matura-Thesis, 2026) und das Arbeitsjournal und eines für den Code (Goth, LifeLike-Engine, 2026). Letzteres ist auch im ersten verlinkt.

8.2 Websites

Bhargava, M., Gabai, D., Kochen, S., & Kohn, J. J. (kein Datum). *John Horton Conway*. Abgerufen am 08. Oktober 2026 von Princeton University: https://www.math.princeton.edu/people/john-horton-conway

dujingning. (01. Mai 2026). *The INI header-only library for Modern C++ supports reading and writing, even writing comments. It is cross-platform and can be used on multiple operating systems.* Abgerufen am 07. Oktober 2026 von inicpp: https://github.com/dujingning/inicpp

GCC. (22. September 2026). *GCC, the GNU Compiler Collection*. Abgerufen am 08. Oktober 2026 von https://gcc.gnu.org/

Geelnard, M., & Löwy, C. (kein Datum). *GLFW is an Open Source, multi-platform library for OpenGL, OpenGL ES and Vulkan development on the desktop.* Abgerufen am 08. März 2026 von GLFW: https://www.glfw.org/

Goth, J. (11. Oktober 2026). *LifeLike-Engine*. Abgerufen am 11. Oktober 2026 von GitHub: https://github.com/j-goth/LifeLike-Engine

Goth, J. (11. Oktober 2026). *LifeLike-Engine Releases*. Abgerufen am 11. Oktober 2026 von GitHub: https://github.com/j-goth/LifeLike-Engine/releases

Goth, J. (11. Oktober 2026). *Matura-Thesis*. Abgerufen am 11. Oktober 2026 von GitHub: https://github.com/j-goth/Matura-Thesis

Khronos Group. (kein Datum-a). *Khronos royalty-free open standards for 3D graphics, Virtual and Augmented Reality, Parallel Computing, Machine Learning, and Vision Processing*. Abgerufen am 08. Oktober 2026 von Khronos Group: https://www.khronos.org/

Khronos Group. (kein Datum-b). *Vulkan Home*. Abgerufen am 08. Oktober 2026 von Vulkan: https://www.vulkan.org/

kitware. (kein Datum). *CMake: A Powerful Software Build System*. Abgerufen am 08. Oktober 2026 von CMake: https://cmake.org/

libsdl-org. (11. Oktober 2026). *SDL*. Abgerufen am 08. Oktober 2026 von https://github.com/libsdl-org/SDL

libsdl-org. (kein Datum). *Simple DirectMedia Layer 3.0*. Abgerufen am 26. Juni 2026 von SDL Wiki: https://wiki.libsdl.org/SDL3/FrontPage

LifeWiki contributors. (15. Dezember 2025). *Conway's Game of Life*. Abgerufen am 08. Oktober 2026 von LifeWiki: https://conwaylife.com/wiki/Conway's_Game_of_Life

Lohmann, N. (11. Oktober 2026). *JSON for Modern C++*. Abgerufen am 07. Oktober 2026 von json: https://github.com/nlohmann/json

LunarG; Khronos Group. (kein Datum). *Welcome to the community for the Vulkan SDK. You can download the latest Vulkan SDK and get SDK questions answered at this site.* Abgerufen am 06. Oktober 2026 von Vulkan: https://vulkan.lunarg.com/home/welcome

Microsoft. (21. August 2026). *Install the Microsoft C++ (MSVC) Build Tools*. Abgerufen am 08. Oktober 2026 von Microsoft Ignite: https://learn.microsoft.com/en-us/cpp/overview/acquire-msvc?view=msvc-170

mingw-w64. (03. Oktober 2026). *mingw-w64*. Abgerufen am 08. Oktober 2026 von mingw-w64: https://www.mingw-w64.org/

MIT. (kein Datum). *The MIT License (MIT)*. Abgerufen am 08. Oktober 2026 von mit-license: https://mit-license.org/

Mohr, T. (kein Datum). *About Me*. Abgerufen am 30. August 2026 von Particle Life Simulator: https://particle-life.com/about/tom-mohr.html

Mohr, T. (kein Datum). *Particle Life Simulator*. Abgerufen am 30. August 2026 von Particle Life Simulator: https://particle-life.com/

OpenAI. (kein Datum). *ChatGPT*. Abgerufen am 11. Oktober 2026 von ChatGPT: https://chatgpt.com/

Overvoorde, A. (2016). *A tutorial that teaches you everything it takes to render 3D graphics with the Vulkan API. It covers everything from Windows/Linux setup to rendering and debugging.* Abgerufen am 04. April 2026 von Introduction - Vulkan Tutorial: https://vulkan-tutorial.com/

Ventrella, J. (Februar 2025). *An asymmetrical particle system*. Abgerufen am 30. August 2026 von Clusters: https://www.ventrella.com/Clusters/

Ventrella, J. (kein Datum). *jeffrey ventrella*. Abgerufen am 30. August 2026 von jeffrey ventrella: http://www.jjventrella.com/

vit-vit. (12. Juni 2015). *Modern and efficient C++ Thread Pool Library*. Abgerufen am 25. August 2026 von CTPL: https://github.com/vit-vit/ctpl

Wikipedia contributors. (14. September 2026). *Lennard-Jones-Potential*. Abgerufen am 31. August 2026 von Wikipedia – Die freie Enzyklopädie: https://en.wikipedia.org/w/index.php?title=Lennard-Jones_potential

8.3 Bildquellen

Abb. 1: https://media.geeksforgeeks.org/wp-content/uploads/20240318035545/game-660.png

Abb. 2: https://hunar4321.github.io/particle-life/images/big_pic.jpg

Abb. 3: Eigene Aufnahme

Abb. 4: Eigene Darstellung

Abb. 5: https://de-academic.com/pictures/dewiki/49/12-6-Lennard-Jones-Potential.png

Abb. 6–12: Eigene Aufnahme

8.4 KI

KI-Modell	Einsatzform	Stelle
ChatGPT (OpenAI, kein Datum)	Erklären der Parameter des Lennar-Jones Potentials	Kapitel 3.5, Absatz nach Abb. 4
ChatGPT (OpenAI, kein Datum)	Erklären der Begriffe im Glossar	Kapitel 7, Glossar

9 Danksagung

Ich bedanke mich bei Herrn Dr. Andreas Leuthardt dafür, dass er mich während der Arbeitsphase betreut, dadurch diese Maturaarbeit ermöglicht, und mir dabei geholfen hat sowohl das Programm als auch die Dokumentation zu verbessern.
Weiterhin danke ich Herrn Adrian Kaiser für die Hilfe bei der Überarbeitung meiner Dokumentation und viele gute Tipps, die mein Programm verbessert haben.
Letztlich möchte ich mich herzlich bei meinen Eltern, Bettina und Markus Goth für die Korrektur der Dokumentation in der letzten Woche vor der Abgabe und viele Verbesserungsvorschläge bedanken.

10 Redlichkeitserklärung

Ich, Jonathan Goth, 4A,
bestätige mit meiner Unterschrift, dass die eingereichte Arbeit selbstständig und ohne unerlaubte Hilfe Dritter verfasst wurde. Die Auseinandersetzung mit dem Thema erfolgte ausschliesslich durch meine persönliche Arbeit und Recherche. Es wurden keine unerlaubten Hilfsmittel benutzt. Ich bestätige, dass ich sämtliche verwendeten Quellen sowie Informanten/-innen im Quellenverzeichnis bzw. an anderer dafür vorgesehener Stelle vollständig aufgeführt habe. Alle Zitate und Paraphrasen (indirekte Zitate) wurden gekennzeichnet und belegt. Sofern ich Informationen von einem KI-System wie bspw. ChatGPT verwendet habe, habe ich diese in meiner Maturaarbeit gemäss den Vorgaben im Leitfaden zur Maturaarbeit korrekt als solche gekennzeichnet, einschliesslich der Art und Weise, wie und mit welchen Fragen die KI verwendet wurde. Ich bestätige, dass das ausgedruckte Exemplar der Maturaarbeit identisch mit der digitalen Version ist. Ich bin mir bewusst, dass die ganze Arbeit oder Teile davon mittels geeigneter Software zur Erkennung von Plagiaten oder KI-Textstellen einer Kontrolle unterzogen werden können.

Datum: 11.10.2026, Unterschrift: 